```python
import torch
import torch.nn as nn

def belief_state_objective(enc_F, enc_B, text_head, x):
  bs, T = x.shape
  forward_state = enc_F(x)
  backward_state = enc_B(x.flip(1)).flip(1)
  ft = torch.arange(T, dtype=torch.int32)
  bt = torch.arange(T, dtype=torch.int32)
  combinations = torch.cartesian_prod(ft, bt)
  combinations = combinations[(combinations[:, 1]-combinations[:, 0]>=2)]
  fb_pairs = combinations.clone()
  fb_pairs = fb_pairs[combinations[:,1] < T]
  f_idxs = fb_pairs[:, 0]
  b_idxs = fb_pairs[:, 1]
  nt_idxs = (combinations[:, 0] + 1)
  f = forward_state[:, f_idxs]
  b = backward_state[:, b_idxs]
  single_labels_f = x[:, nt_idxs].unsqueeze(2)
  single_labels_b = x[:, b_idxs].unsqueeze(2)
  single_labels = torch.cat((single_labels_f, single_labels_b), dim=2)
  logits = text_head(torch.cat([f, b],dim=2))
  fb_numpairs = fb_pairs.shape[0]
  logits = logits.reshape((bs, fb_numpairs, 2, -1))
  logits = logits.reshape((bs*fb_numpairs*2, -1))
  single_labels = single_labels.reshape((bs*fb_numpairs*2))
  loss = nn.CrossEntropyLoss()(logits, single_labels)
  return loss

if __name__ == '__main__':
  batch_size = 8
  T = 12
  m = 512
  num_tokens = 100
  #Use dummy function in place of actual autoregressive transformer
  enc_F = nn.Sequential(nn.Embedding(num_tokens, m), nn.Linear(m, m))
  enc_B = nn.Sequential(nn.Embedding(num_tokens, m), nn.Linear(m, m))
  text_head = nn.Sequential(nn.Linear(m*2, m), nn.LeakyReLU(), nn.Linear(
                                    m, num_tokens*2))
  x = torch.randint(0, num_tokens, size=(batch_size,T))

  loss = belief_state_objective(enc_F, enc_B, text_head, x)
  print(loss)
```

Figure 12: A simple implementation of the belief state transformer objective

```
import torch
import torch.nn as nn

def belief_state_objective(all_f, all_b, text_head, x):
  bs, T = x.shape
  forward_state = all_f
  backward_state = all_b.flip(1)
  ft = torch.arange(T, dtype=torch.int32)
  bt = torch.arange(T, dtype=torch.int32)
  combinations = torch.cartesian_prod(ft, bt)
  combinations = combinations[(combinations[:, 1]-combinations[:, 0]>=2)]
  fb_pairs = combinations.clone()
  fb_pairs = fb_pairs[combinations[:,1] < T]
  f_idxs = fb_pairs[:, 0]
  b_idxs = fb_pairs[:, 1]
  nt_idxs = (combinations[:, 0] + 1)
  f = forward_state[:, f_idxs]
  b = backward_state[:, b_idxs]
  single_labels_f = x[:, nt_idxs].unsqueeze(2)
  single_labels_b = x[:, b_idxs].unsqueeze(2)
  single_labels = torch.cat((single_labels_f, single_labels_b), dim=2)
  logits = text_head(torch.cat([f, b],dim=2))
  fb_numpairs = fb_pairs.shape[0]
  logits = logits.reshape((bs, fb_numpairs, 2, -1))
  logits = logits.reshape((bs*fb_numpairs*2, -1))
  single_labels = single_labels.reshape((bs*fb_numpairs*2))
  loss = nn.CrossEntropyLoss()(logits, single_labels)
  return loss

if __name__ == '__main__':
  batch_size = 8
  T = 12
  m = 512
  num_tokens = 100
  #Use dummy function in place of actual autoregressive transformer
  enc_F = nn.Sequential(nn.Embedding(num_tokens, m), nn.Linear(m, m))
  enc_B = nn.Sequential(nn.Embedding(num_tokens, m), nn.Linear(m, m))
  text_head = nn.Sequential(nn.Linear(m*2, m), nn.LeakyReLU(), nn.Linear(
                                        m, num_tokens*2))
  x = torch.randint(0, num_tokens, size=(batch_size,T))
  f = enc_F(x)
  b = enc_B(x)

  # just get the text head computation graph
  _f = f.detach()
  _b = b.detach()
  _f.requires_grad = True
  _b.requires_grad = True

  loss = belief_state_objective(_f, _b, text_head, x)
  # compute text head gradients over all prefix/suffix pairs.
  loss.backward()
  # Update encoders with 1 backward pass.
  f.backward(_f.grad)
  b.backward(_b.grad)
```

Figure 13: Efficient computation of all prefix-suffix losses.